TAKE-HOME ASSIGNMENT

ezeebit

Backend Engineer · Mid to Senior

Secure. Speedy. Ezee.

About Ezeebit

Ezeebit is Africa's leading crypto and stablecoin payment infrastructure company. We let merchants accept digital assets in-store and online, hedge instantly, and settle in their local currency the next day. We are FSCA-regulated (CASP FSP No. 53664), based in Cape Town, and backed by Founder Collective, Raba Partnership, and DCG.

We work with leading retail brands in-store — including iStore, Diesel, Le Creuset, Amiri, and Goodman Gallery — and we are now expanding fast into online commerce. We have begun processing for online merchants such as Sunbet, partnered with payment service provider Callpay to reach more online businesses, and built a Shopify payments app for e-commerce stores.

We are a small, fast-moving team building the financial infrastructure for a continent, which means the engineers we hire own real problems end to end.

About this assignment

The exercise that follows is a slice of that real work: the service that holds, converts, and moves merchants' money. It is deliberately open-ended. There is more than one good answer, and we are most interested in how you design it.

The brief

Our merchants operate in different countries and hold money in both their local fiat currency (for example ZAR, NGN, or KES) and stablecoins (for example USDT or USDC). They need to hold balances, convert between a fiat currency and a stablecoin, and move money in and out — and they do this from a dashboard that fires requests over flaky mobile connections, sometimes more than once.

Volume is moderate today but growing fast, with bursts around paydays and big retail events. The money in these wallets is real: a single lost, duplicated, or mis-converted transaction is a serious incident, so correctness and a clear audit trail matter.

What we are looking for

We are most interested in your **design**: the services, endpoints, and functions you would build, and the reasoning behind them. A working implementation of your chosen tasks is welcome, but you do not need to build out a full, production-ready codebase — spend your time on the parts that best show your thinking.

Assume the surrounding infrastructure already exists: blockchain confirmation services are up and running, and third-party integrations such as exchange-rate feeds are complete. You can call these services rather than build them.

We will be looking at:

- **Design and structure** — clear services, endpoints, and data models, and how they fit together.
- **Safety with transactions and money** — balances and transactions that stay correct under duplicate requests, failures, and load.
- **Communication** — how clearly you explain your decisions, in writing and on the walkthrough.

How to approach it

Below are six possible tasks. Choose **two or three** that you can complete well in the time available — building out all six is not necessary. When you describe your solution, show how the tasks you chose fit together, and briefly how the remaining tasks would slot into the same design.

Choose two or three of the following tasks

TASK 1

Hold a merchant's money across currencies

Build the part of the service that keeps track of what each merchant holds. A merchant can hold balances in several currencies at once — at least one local fiat currency and at least one stablecoin. Money arrives (a deposit) and the balance goes up. We need to trust these balances completely and be able to explain, at any later date, exactly how a balance reached its current value. Assume amounts can be large, and that two different currencies must never be silently mixed.

TASK 2

Convert one currency into another

Let a merchant convert an amount of one currency they hold into another (for example, sell USDT for ZAR, or buy USDT with ZAR). You will need a price; assume an exchange-rate service already exists that you can call for the current rate, and that it can be slow and its price keeps moving. Think about which rate the merchant actually gets, what happens between quoting a price and completing the conversion, and how you make sure the platform does not end up on the wrong side of a moving market. A merchant must never be able to convert more than they actually hold.

TASK 3

Withdraw funds out of the platform

Let a merchant request a payout of a currency they hold to an external destination (a bank account for fiat, a blockchain address for a stablecoin). Assume the payout rail already exists and is asynchronous: you ask it to pay out, and some time later it tells you whether it succeeded or failed. The merchant's app may send the same withdrawal more than once, and merchants may fire several at the same time. Make sure a merchant can never withdraw money they do not have, the same withdrawal is never paid out twice, and the balance is always right — even when a payout ultimately fails.

TASK 4

Accept an incoming crypto payment

A customer pays a merchant in stablecoin, and the funds arrive on a blockchain. Build the flow that turns an incoming on-chain payment into balance the merchant can actually use. Assume the blockchain confirmation service notifies you when funds arrive, and notifies you again as the payment becomes more final over time — and that it may send you the same notification more than once. Decide when a payment is safe to treat as money the merchant can spend, and make sure a single payment is never counted twice.

TASK 5

Auto-settle incoming crypto to local currency

Many merchants do not want to hold crypto — they want their local currency, and quickly. Let a merchant set a simple rule such as “whenever I receive a stablecoin, automatically convert a set percentage of it to my local currency.” Build the part of the service that applies this rule as money comes in. Think about what the merchant should see, what happens if the conversion cannot complete straight away, and how to keep the outcome predictable and easy to explain.

TASK 6

Route payouts across countries and partners

We pay merchants out in different countries, and each country has its own banking and payout partners with their own rules, limits, and occasional downtime. Design how a single withdrawal request is routed to the right payout partner for the merchant’s country, and what happens when the chosen partner is slow, unavailable, or rejects the request. Assume the partner integrations already exist — we are interested in how you structure the routing and keep a merchant’s money safe and accounted for when a partner fails.

Constraints

- Use an **object-oriented language and a relational database**. Our own stack is Java with Spring Boot and MySQL, so that is a natural choice, but any mainstream OO language and relational database is fine — just justify non-trivial picks.
- You do not need to integrate real third-party services or move real funds. Assume the surrounding services (exchange rates, blockchain confirmations, payout partners) already exist and can be called.
- No user interface is needed, and no real authentication or KYC — assume a merchant is already identified by an ID in the request.
- Correctness of transactions and money handling is of importance, as we deal with merchant funds in real-world payment systems. Anything you do not finish, capture in your SOLUTION.md.

What to submit

A single GitHub repository containing:

- **README.md** — how to run it locally.
- **SOLUTION.md** — your design and the trade-offs you made, which tasks you chose, how the remaining tasks would fit into the same design, and what you would do with more time.

- **A walkthrough video of around 10 minutes** (Loom or similar) — walk us through your design and what you built, your key decisions, and anything you would change.
- **AI usage notes** — where you used AI tools, and where you accepted, changed, or rejected what they produced.

Time

Aim for around **two to three hours**. If you go over, stop and write the rest up in SOLUTION.md — we would rather see your thinking than a rushed implementation. Strong candidates are usually juggling several of these at once, so we have scoped this to respect your time.

A few notes

- Using AI tools is welcomed, not penalised. We build with them every day; we want to see how you direct them and where your judgement overrides theirs.
- Even though these services are available, real dependencies still fail. Show how you would handle incorrect data or an unreliable connection to one of them — for example a bad exchange rate, a timeout, or a duplicate notification.
- If something is genuinely ambiguous, make a reasonable assumption, note it, and move on. Spotting the ambiguity is part of the exercise.

> We are excited to see how you approach this. If anything is blocking you, reply to our email or reach us at **saul@ezeebit.com**.